Infrastructure Description

AWS Services

Udagram uses the following AWS services:

Service	Purpose
Amazon S3	Hosts the compiled Ionic/Angular frontend as a static website. The same deployment also uses S3 object storage for application image files.
AWS Elastic Beanstalk	Hosts the Node.js/Express backend API. The API is deployed from the compiled backend bundle.
Amazon RDS	Hosts the PostgreSQL database used by Sequelize models for users and feed items.
IAM	Provides deployment credentials used by CircleCI to publish builds to S3 and Elastic Beanstalk.

Deployed Resources

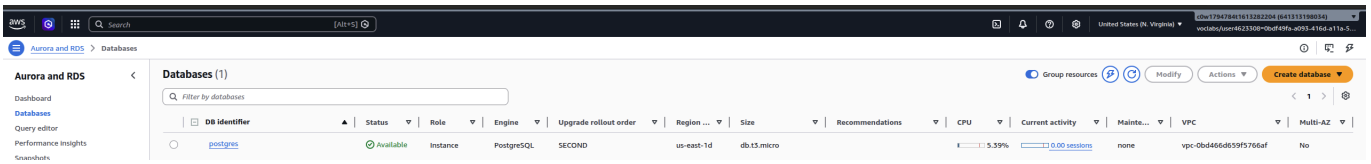
- Frontend S3 bucket: `mj-udacity-udagram-s3`
- Frontend URL: `http://mj-udacity-udagram-s3.s3-website-us-east-1.amazonaws.com`
- API Elastic Beanstalk environment: `Mj-udacity-udagram-api-env`
- API URL: `http://mj-udacity-udagram-api-env.eba-wcfdcn9n.us-east-1.elasticbeanstalk.com/api/v0`
- Database: PostgreSQL on Amazon RDS

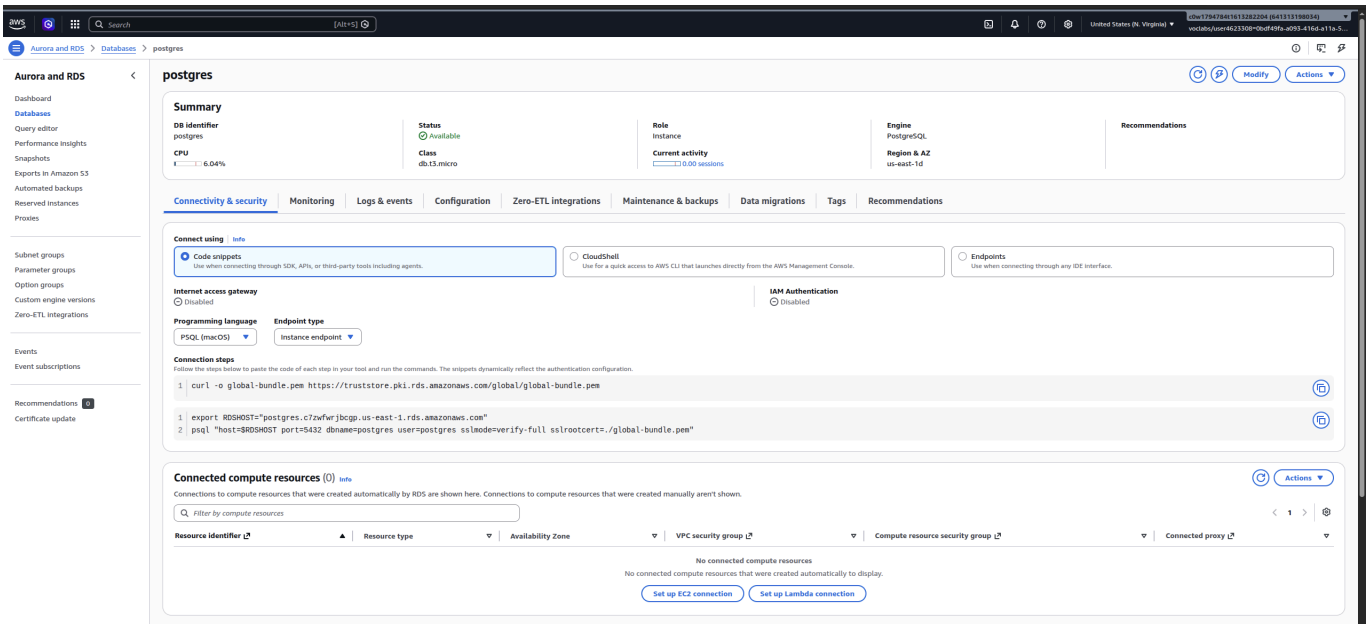
Architecture Diagram

See Architecture.md.

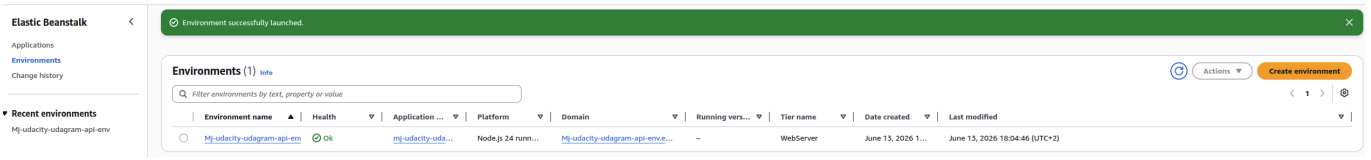
Infrastructure Screenshots

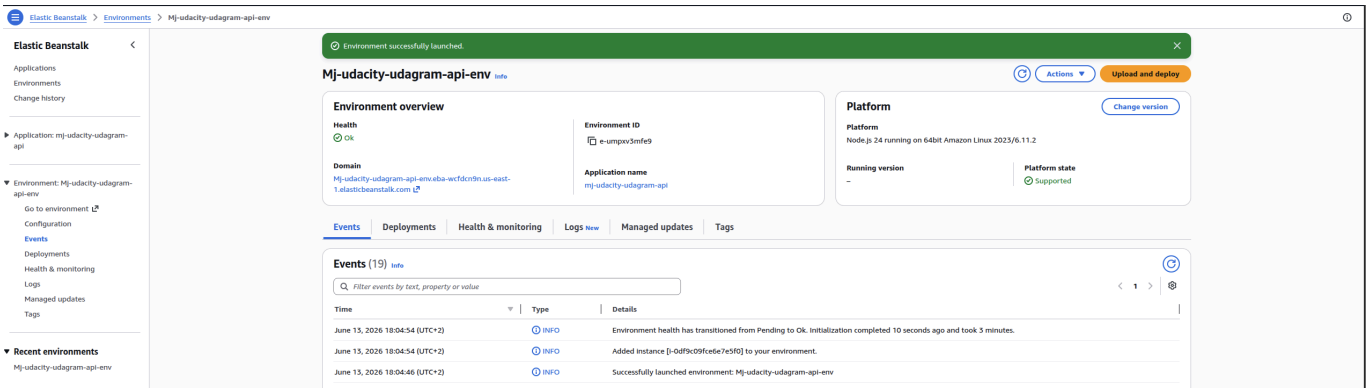
RDS



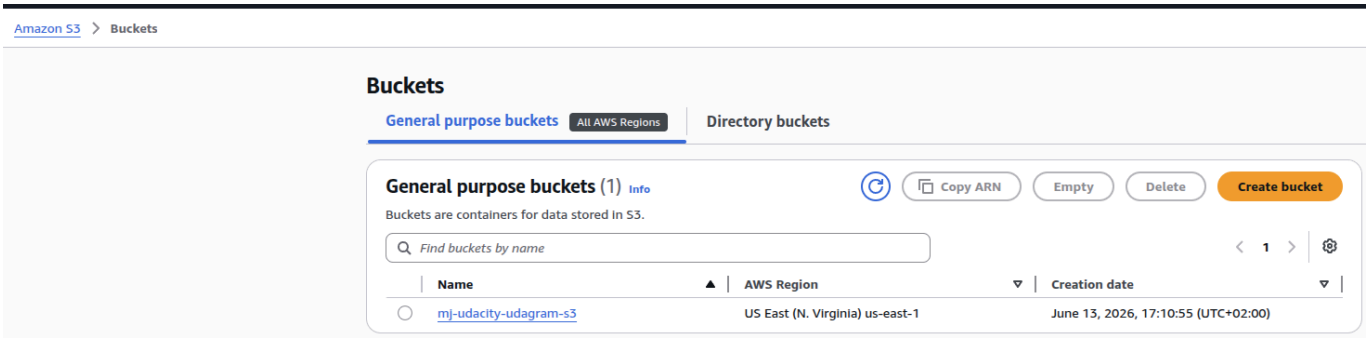


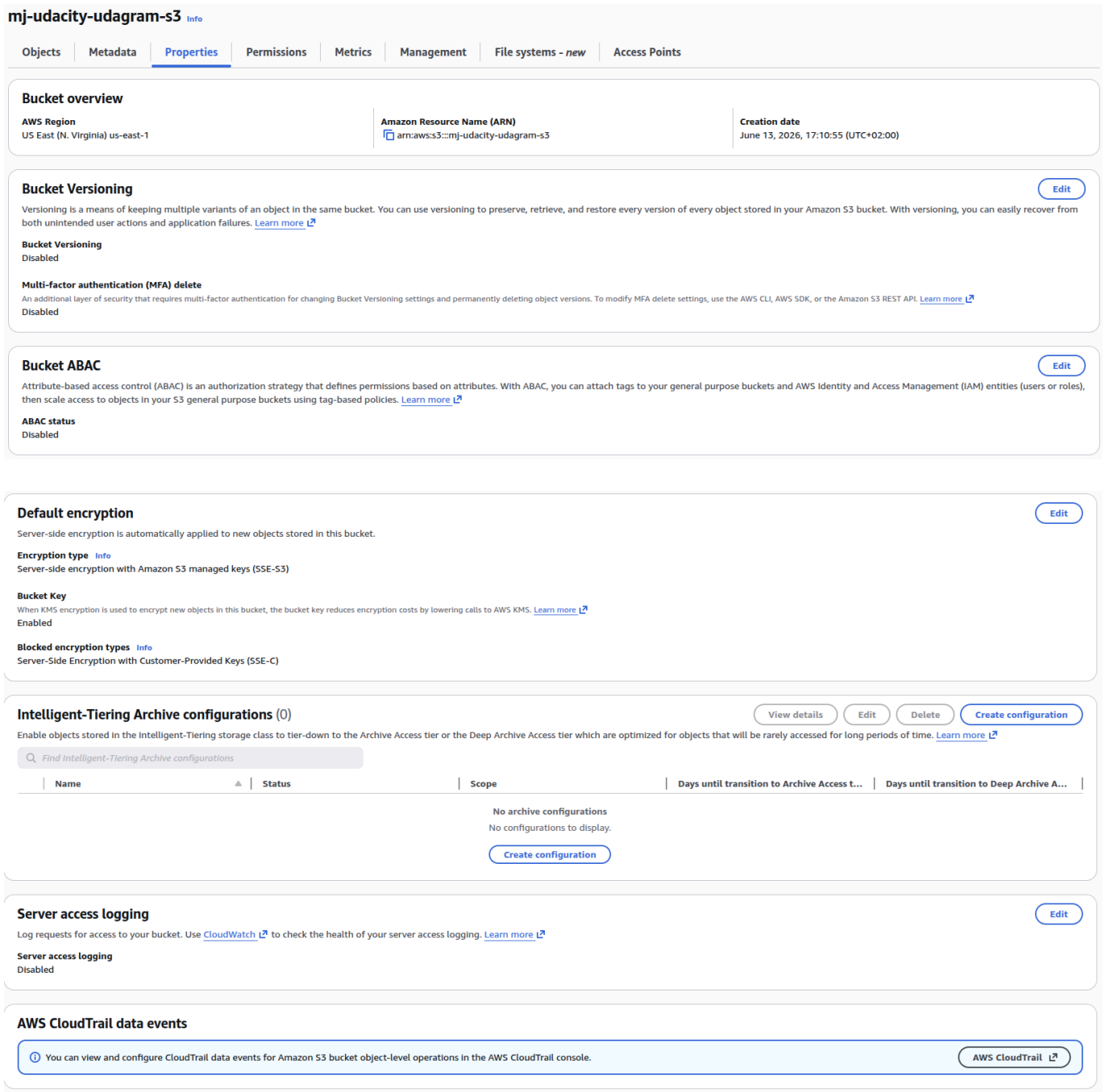
Elastic Beanstalk





S3





mj-udacity-udagram-s3 Info

Objects | Metadata | Properties | Permissions | Metrics | Management | File systems - *new* | Access Points

Bucket overview

AWS Region
US East (N. Virginia) us-east-1

Amazon Resource Name (ARN)
arn:aws:s3:::mj-udacity-udagram-s3

Creation date
June 13, 2026, 17:10:55 (UTC+02:00)

Bucket Versioning Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. Learn more

Bucket Versioning
Disabled

Multi-factor authentication (MFA) delete
An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. Learn more
Disabled

Bucket ABAC Edit

Attribute-based access control (ABAC) is an authorization strategy that defines permissions based on attributes. With ABAC, you can attach tags to your general purpose buckets and AWS Identity and Access Management (IAM) entities (users or roles), then scale access to objects in your S3 general purpose buckets using tag-based policies. Learn more

ABAC status
Disabled

Default encryption Edit

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type Info
Server-side encryption with Amazon S3 managed keys (SSE-S3)

Bucket Key
When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering calls to AWS KMS. Learn more
Enabled

Blocked encryption types Info
Server-Side Encryption with Customer-Provided Keys (SSE-C)

Intelligent-Tiering Archive configurations (0) View details | Edit | Delete | Create configuration

Enable objects stored in the Intelligent-Tiering storage class to tier-down to the Archive Access tier or the Deep Archive Access tier which are optimized for objects that will be rarely accessed for long periods of time. Learn more

Find Intelligent-Tiering Archive configurations

Name	Status	Scope	Days until transition to Archive Access t...	Days until transition to Deep Archive A...

No archive configurations
No configurations to display.
Create configuration

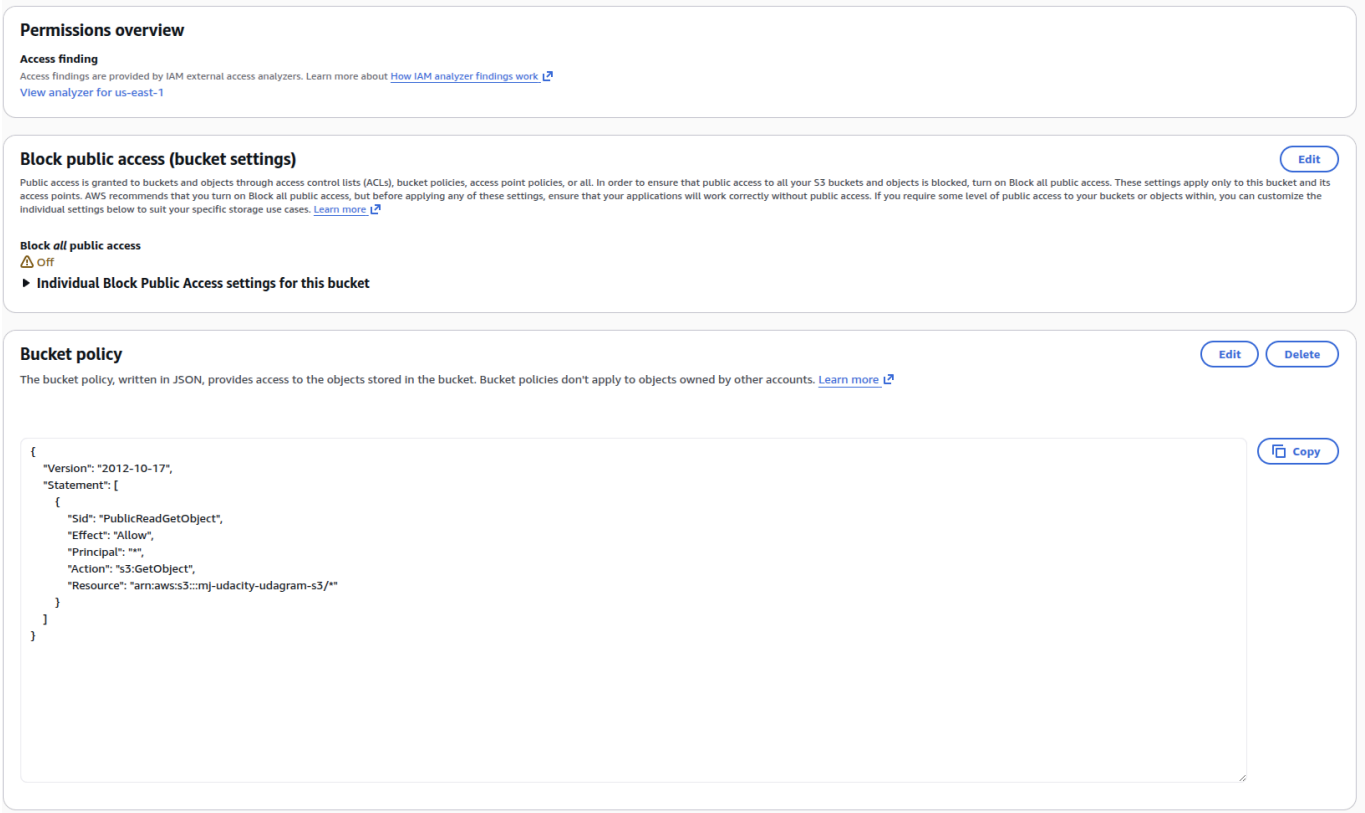
Server access logging Edit

Log requests for access to your bucket. Use CloudWatch to check the health of your server access logging. Learn more

Server access logging
Disabled

AWS CloudTrail data events

You can view and configure CloudTrail data events for Amazon S3 bucket object-level operations in the AWS CloudTrail console. AWS CloudTrail



Environment Configuration

Environment-specific and secret values are not committed to source code. The production API loads configuration from environment variables through `udagram/udagram-api/src/config/config.ts`. CircleCI stores the required secrets as project environment variables for deployment, and the deploy job runs `eb setenv` to copy the backend runtime values into the Elastic Beanstalk environment properties. The running API process reads those values from Elastic Beanstalk at runtime, not from checked-in files.

Production values include:

- `POSTGRES_HOST`
- `POSTGRES_DB`
- `POSTGRES_USERNAME`
- `POSTGRES_PASSWORD`
- `AWS_BUCKET`
- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_REGION`
- `JWT_SECRET`
- `URL`

The variable names must match the backend configuration exactly. Use `POSTGRES_USERNAME`, not `POSTGRES_USER`; use `AWS_REGION`, not `AWS_DEFAULT_REGION`.

Elastic Beanstalk production environment properties should include:

- `POSTGRES_HOST`
- `POSTGRES_DB`
- `POSTGRES_USERNAME`

- POSTGRES_PASSWORD
- AWS_BUCKET
- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_REGION
- JWT_SECRET
- URL